



**Design and Implementation of an eBPF-based Runtime Security
Monitoring Tool**

Candidate: Simone Romano (s344024)
Supervisor: Prof. Riccardo Sisto
Company tutor: Dr. David Soldani
Academic year: 2025–2026

Summary

Context and objective. Containers and orchestration platforms require applications to be observed without modifying their code or relying solely on their logs. eBPF allows verified programs to be attached to selected points in the Linux kernel, exposing processes, credentials, files, memory and isolation mechanisms. This thesis designs and implements a runtime monitoring prototype that selectively collects host-level events and transfers them to user space with the context required for analysis.

The reference environment is Rocky Linux 8.10 with kernel 4.18. Compatibility is verified on the actual kernel because CO-RE and BTF adapt structure access but cannot provide missing hooks, helpers or attachment mechanisms. The prototype observes and detects events without changing kernel decisions.

Host-level hook implementation. The main contribution is the selection and implementation of the host hooks. The eBPF programs attach to system calls, tracepoints, kernel functions and Linux Security Module interfaces. A system-call entry preserves the request, an LSM hook observes security mediation, a tracepoint may confirm a transition, and a return hook adds the reported result.

Direct hooks produce an event in one invocation. Paired hooks instead retain entry arguments and complete them at return, using temporary state identified by thread and event. Per-CPU workspaces keep large records off the eBPF stack. Arguments are serialised with explicit indexes and bounds, and only the populated record prefix crosses the perf-event buffer.

Process hooks distinguish a request from the task's actual lifecycle. Fork, vfork and clone retain the system-call result, while scheduler tracepoints describe creation, executable replacement and exit. For execution, entry hooks preserve the path and arguments, *security_bprm* hooks add the resolved binary identity, and *sched_process_exec* confirms a successful transition. A separate event records a negative execve or execveat result.

Identity hooks similarly separate request, mediation and installed state. The *setuid* and *setgid* families record identifiers and results; *security_task_fix_setuid* observes the proposal, whereas *commit_creds* compares old and new credentials when they become active. UIDs, GIDs, the user namespace and capability sets expose the transition without confusing an intermediate check with its final outcome.



Filesystem hooks distinguish the pathname supplied by the process from the object resolved by the kernel. The open, chmod and chown system calls describe request and result, combining several variants in a shared schema. *security_file_open* instead records the resolved path, device, inode and change time, while retaining the caller pathname when recoverable. Further hooks observe permissions, ioctl, directory-entry changes and mount topology.

Coverage also includes memory, cross-process transfers, namespaces and cgroups. The mmap and mprotect system calls preserve requests and results, while security hooks add the kernel objects involved. *switch_task_ns* records only changed identifiers, and cgroup hooks use local kernel identities. Kernel-facing surfaces include modules, eBPF operations, kprobes, symbols, procfs, user-mode helpers and files read by the kernel. These events provide evidence to detectors without independently proving authorisation or integrity.

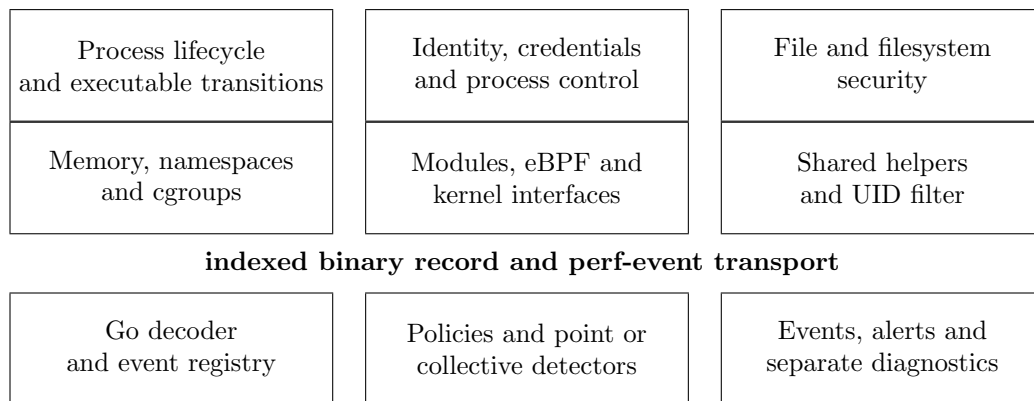


Figure 1: Host-level hook families and their shared path to user-space analysis.

The hooks share initialisation, UID filtering, serialisation and submission. Their common context preserves process identity, namespaces, timestamps, system calls and logical events. Explicit CO-RE branches handle differences in the target kernel, while loops and variable accesses obey verifier-visible bounds.

User-space processing. The Go runtime selects programs, expands dependencies and decodes records into typed events through a shared registry. Policies define scope and suppression; deterministic detectors recognise point conditions or short collective sequences correlated by process, file or cgroup. Alerts preserve the matching evidence and may include MITRE ATT&CK classification metadata.

Validation and results. The Minikube validation separated the monitor, AES-beta target and client into three Pods on one node. Three benign runs produced no alerts; three exploit runs recovered the synthetic flag and generated the expected sequence. This confirms node-local visibility in the controlled scenario, without implying general attack coverage or cross-node monitoring.